Pokročilé databázové systémy – semestrálna práca

AUTOSERVIS

Informačný systém autoservisu – Oracle, PL/SQL, objekty a kolekcie, XML / JSON, Java Swing

Inštalačná a technická príručka

Údaj	Hodnota
Tím	Člen 1 – AnatoliuLop (A), člen 2 – Matej Paprnák (M), člen 3 – Danyil Berezhnyi (D)
Repozitár	https://github.com/Ri4ik/autoservis-pds (pôvodný: github.com/AnatoliuLop/autoservis-pds)
Databáza	Oracle 21c, školský server oradb.fri.uniza.sk
Verzia dokumentu	október 2026 – checkpoint (5. týždeň)

Obsah

1. Čo je projekt Autoservis 3
1.1 Čo nájdete v tejto príručke 3
2. Požiadavky 3
3. Stiahnutie projektu 3
3.1 Štruktúra priečinkov 4
4. Príprava databázy 4
4.1 Pripojenie v SQL Developeri 4
4.2 Nová (prázdna) schéma – poradie skriptov 4
4.3 Existujúca schéma zo staršej verzie 5
4.4 Kontrola, že je databáza pripravená 5
4.5 Generovanie väčšieho množstva údajov 5
5. Zostavenie a spustenie aplikácie 6
5.1 Zostavenie cez Maven 6
5.2 Zostavenie a spustenie v IntelliJ IDEA 6
5.3 Spustenie a prihlásenie 6
6. Ako je všetko prepojené – architektúra 6
6.1 Kto s kým komunikuje 7
7. Dátový model 7
7.1 Tabuľky a ich vzťahy 7
7.2 Pravidlá, ktoré stráži databáza 8
7.3 Priebeh servisnej zákazky 8
7.4 ER diagram 9
8. Logika v databáze 10
8.1 Objektové typy a kolekcie 10
8.2 PL/SQL procedúry a funkcie 10
8.3 Pohľady a reporty 11
8.4 XML a JSON 12
9. Aplikácia 12
9.1 Záložky a ich prepojenie na databázu 12
9.2 Triedy aplikácie 12
9.3 Ako aplikácia pracuje s databázou 13
10. Priebeh typických scenárov 13
10.1 Nová zákazka a jej dokončenie 13
10.2 Report 13
10.3 Export XML s XPath 13
11. Riešenie problémov 14
12. Rozdelenie práce v tíme 14

1. Čo je projekt Autoservis

Projekt je informačný systém autoservisu. Skladá sa z dvoch častí, ktoré spolu úzko súvisia:

- **Databáza v Oracle** – tabuľky, obmedzenia, objektové typy, kolekcie, pohľady a PL/SQL procedúry. Vytvára sa SQL skriptmi `01_…` až `16_…` v SQL Developeri. Tu je celá logika a všetky pravidlá nad dátami.
- **Desktopová aplikácia v Jave** (priečinok `app/`) – grafické rozhranie, ktoré sa cez JDBC pripojí k tej istej schéme a sprístupňuje dáta, procedúry, reporty a exporty. Aplikácia nemá vlastnú databázu ani vlastné pravidlá – všetko číta z Oracle a volá jeho objekty.

Systém eviduje zákazníkov a ich vozidlá, servisné zákazky s kontrolným zoznamom pri prijatí, vykonané servisné úkony a použité náhradné diely, faktúry, platby a prílohy (fotografie, dokumenty ako BLOB). Nad dátami sú reporty (agregácie, analytické funkcie, časové okná), XML a JSON exporty a objektové typy s metódami.

1.1 Čo nájdete v tejto príručke

- kapitoly 2 – 5: **inštalácia a nastavenie** – čo nainštalovať, ako vytvoriť databázu, ako zostaviť a spustiť aplikáciu,
- kapitoly 6 – 9: **ako to celé funguje** – architektúra, dátový model, objekty a kolekcie, PL/SQL, reporty, XML / JSON a aplikácia,
- kapitola 10: **priebeh typických scenárov** krok za krokom (čo sa deje v aplikácii aj v databáze),
- kapitoly 11 – 12: **riešenie problémov** a rozdelenie práce v tíme.

2. Požiadavky

Čo	Verzia / poznámka	Na čo
Oracle Database	19c alebo novšia (školský server je 21c)	databáza projektu; JSON_SERIALIZE vyžaduje 19c+
Práva v schéme	CREATE TABLE, CREATE VIEW, CREATE TYPE, CREATE PROCEDURE	vytvorenie všetkých objektov skriptami
Oracle SQL Developer	ľubovoľná aktuálna (overené 23.1)	spúšťanie SQL skriptov (F5), Data Modeler
Java (JDK)	17 alebo novšia (overené 21 a 25)	zostavenie a spustenie aplikácie
Maven alebo IntelliJ IDEA	Maven 3.9+ / IntelliJ s podporou Maven	zostavenie aplikácie (stiahne Oracle JDBC ovládač)
Git	voliteľne	stiahnutie repozitára (alebo ZIP z GitHubu)

Pozor: Aplikácia aj skripty pracujú v **jednej schéme** (vaše používateľské meno na školskom serveri). Každý člen tímu môže mať vlastnú schému – stačí v nej spustiť skripty a pri prihlásení do aplikácie zadať svoje meno.

3. Stiahnutie projektu

Projekt je na GitHube. Možnosti stiahnutia:

1. **Git:** v príkazovom riadku spustite `git clone https://github.com/Ri4ik/autoservis-pds.git`.
2. **ZIP:** na stránke repozitára kliknite na zelené tlačidlo **Code → Download ZIP** a rozbaľte ho, napr. do `C:\Projects\autoservis-pds`.

3.1 Štruktúra priečinkov

Súbor / priečinok	Obsah
`01_schema.sql`	objektové typy kolekcií a 11 tabuliek s kľúčmi, obmedzeniami a indexmi
`02_demo_data.sql`	demonštračné údaje (45 záznamov) vrátane histórie km a kontrolných zoznamov
`02a_migracia_kolekcie.sql`	iba pre schému vytvorenú staršou verziou `01_schema.sql` (bez kolekcií)
`03_validation_reports.sql`, `04_reports.sql`	kontrola údajov a reporty 1, 2, 5, 9
`05_object_type.sql`, `08_object_servisna_polozka.sql`	objektové typy T_ZAKAZNIK a T_SERVISNA_POLOZKA
`09_…` až `16_…`	XML report, reporty, PL/SQL procedúry, generátor údajov, exporty, reporty nad kolekciami
`Autoservis_ERD_v2.png`	aktuálny ER diagram (pôvodný je `Autoservis_ERD.png`)
`app/`	Java aplikácia: `pom.xml`, zdrojové kódy `src/main/java/sk/autoservis/`, `spusti.bat`, `spusti.sh`
`tools/ErdGen.java`	generátor ER diagramu z dátového slovníka
`README.md`	stručná dokumentácia, rozdelenie práce a stav plnenia

4. Príprava databázy

4.1 Pripojenie v SQL Developeri

1. Spustite **Oracle SQL Developer** a v paneli **Connections** kliknite na zelené **+**.
2. Vyplňte: **Connection Name** `Autoservis`, **Username** a **Password** – vaše školské prihlasovacie údaje, **Hostname** `oradb.fri.uniza.sk`, **Port** `1521`, **Service name** `orclpdb.fri.uniza.sk`.
3. Kliknite **Test** – musí sa zobraziť *Status: Success* – a potom **Connect**.

4.2 Nová (prázdna) schéma – poradie skriptov

Každý skript otvorte cez **File → Open**, vyberte pripojenie a spustite celý skript klávesom **F5** (Run Script). Poradie je dôležité – neskoršie skripty používajú objekty zo starších.

#	Skript	Čo vytvorí	Potrebné pre
1	`01_schema.sql`	typy T_STAV_KM, T_HISTORIA_KM, T_BOD_KONTROLY, T_KONTROLNY_ZOZNAM; 11 tabuliek	všetko
2	`02_demo_data.sql`	demo údaje vrátane kolekcií, COMMIT	všetko
3	`03_validation_reports.sql`	iba kontrolné SELECT-y (nič nemení)	kontrola
4	`04_reports.sql`	iba SELECT-y reportov 1, 2, 5, 9	kontrola
5	`05_object_type.sql`	typ T_ZAKAZNIK (aplikácia ho vie vytvoriť aj sama)	Objekty a kolekcie
6	`08_object_servisna_polozka.sql`	typ T_SERVISNA_POLOZKA, tabuľka SERVISNA_POLOZKA_ZAKAZKY, dáta	Objekty a kolekcie
7	`09_xml_report_zakazka.sql`	pohľad V_XML_ZAKAZKA, tabuľka XML_ZAKAZKY OF XMLTYPE	XML
8	`10_report8_klzavy_priemer.sql`	pohľad V_KLZAVY_PRIEMER_TRZIEB	Reporty
9	`11_report10_historia_vozidla.sql`	typy histórie, funkcia HISTORIA_VOZIDLA, procedúra VYPIS_HISTORIU_VOZIDLA	História vozidla, XML, JSON
10	`12_plsql_zakazky.sql`	procedúry zákaziek, PRIDAJ_STAV_KM, STANDARDNY_KONTROLNY_ZOZNAM	Zákazky, Objekty a kolekcie
11	`13_analyticke_reporty.sql`	pohľady reportov 3, 4, 6, 7	Reporty
12	`14_generator_dat.sql`	procedúra GENERUJ_TESTOVACIE_DATA	generovanie údajov

#	Skript	Čo vytvorí	Potrebné pre
13	`15_exporty_xml_json.sql`	pohľady V_JSON_ZAKAZKA, V_XML / V_JSON_HISTORIA_VOZIDLA	XML, JSON
14	`16_kolekcie_reporty.sql`	pohľady reportov 13, 13b, 14, 14b	Reporty

Tip: Testovacie bloky na konci skriptov (ukážky procedúr, generátora, dočasné dáta histórie) končia príkazom `ROLLBACK`, takže demo údaje zostanú nezmenené. Vo výstupe skriptu (Script Output) vidíte výsledky aj očakávané hodnoty v komentároch `OCAKAVANE`.

4.3 Existujúca schéma zo staršej verzie

Ak ste tabuľky vytvorili ešte pôvodným `01_schema.sql` (bez stĺpcov `historia_km` a `kontrolny_zoznam`), schému **nemažte** – stačí ju doplniť:

1. spustite `02a_migracia_kolekcie.sql` – vytvorí typy, pridá stĺpce (`ALTER TABLE … ADD`), doplní demo hodnoty a prekompiluje závislé objekty,
2. potom spustite (znova) skripty `09`, `12`, `14`, `15` a nový `16` – obsahujú `CREATE OR REPLACE`, takže prepíšu staršie verzie.

Pozor: Skripty `01`, `02`, `08` a časť `09` (tabuľka XML_ZAKAZKY) vytvárajú tabuľky príkazom `CREATE TABLE` – pri opakovanom spustení skončia chybou *ORA-00955 name is already used*. To je v poriadku; na úplné vyčistenie slúži časť **UPRATANIE** na konci každého skriptu (zakomentované príkazy DROP).

4.4 Kontrola, že je databáza pripravená

```
-- 11 tabuliek + SERVISNA_POLOZKA_ZAKAZKY a XML_ZAKAZKY
SELECT table_name FROM user_tables ORDER BY 1;

-- objekty projektu tu nesmú byť (objekty z cvičení nevadia)
SELECT object_name, object_type FROM user_objects WHERE status = 'INVALID';

-- kolekcie v demo údajoch: očakávané 5 a 5
SELECT COUNT(*) FROM vozidlo WHERE historia_km IS NOT NULL;
SELECT COUNT(*) FROM servisna_zakazka WHERE kontrolny_zoznam IS NOT NULL;
```

4.5 Generovanie väčšieho množstva údajov

Demo údaje (45 záznamov) stačia na overenie funkčnosti. Pre zmysluplné testovanie a neskôr analýzu indexov slúži generátor:

```
EXEC generuj_testovacie_data(100);    -- cca 100 zákazníkov, ~1 200 záznamov
COMMIT;
-- finálna práca, analýza indexov:
EXEC generuj_testovacie_data(10000);  -- cca 120 000 záznamov
COMMIT;
```

Generátor dodržiava všetky pravidlá modelu: dátumy idú po sebe, tachometer rastie, faktúra = úkony + diely, odovzdaná zákazka je zaplatená, vozidlá dostanú históriu km a zákazky kontrolný zoznam. Generované záznamy majú ID od 1000, demo údaje (ID 1 – 999) zostanú nedotknuté.

Pozor: Po trvalom vygenerovaní sa zmenia výsledky reportov – očakávané hodnoty v komentároch skriptov platia len pre demo údaje. Generujte preto v schéme, ktorú budete prezentovať, až keď sa na tom tím dohodne.

5. Zostavenie a spustenie aplikácie

5.1 Zostavenie cez Maven

```
cd app
mvn clean package
```

Maven stiahne závislosti (Oracle JDBC ovládač `ojdbc11`) a vytvorí jeden spustiteľný súbor `app/target/autoservis-pds.jar`, ktorý obsahuje všetko potrebné vrátane skriptu `05_object_type.sql`.

5.2 Zostavenie a spustenie v IntelliJ IDEA

1. **File → Open** a vyberte súbor `app/pom.xml` → **Open as Project**.
2. Počkajte, kým IntelliJ načíta Maven projekt (stiahne závislosti).
3. Otvorte triedu `sk.autoservis.Main` a kliknite na zelenú šípku **Run**. JAR môžete zostaviť v paneli **Maven → Lifecycle → package**.

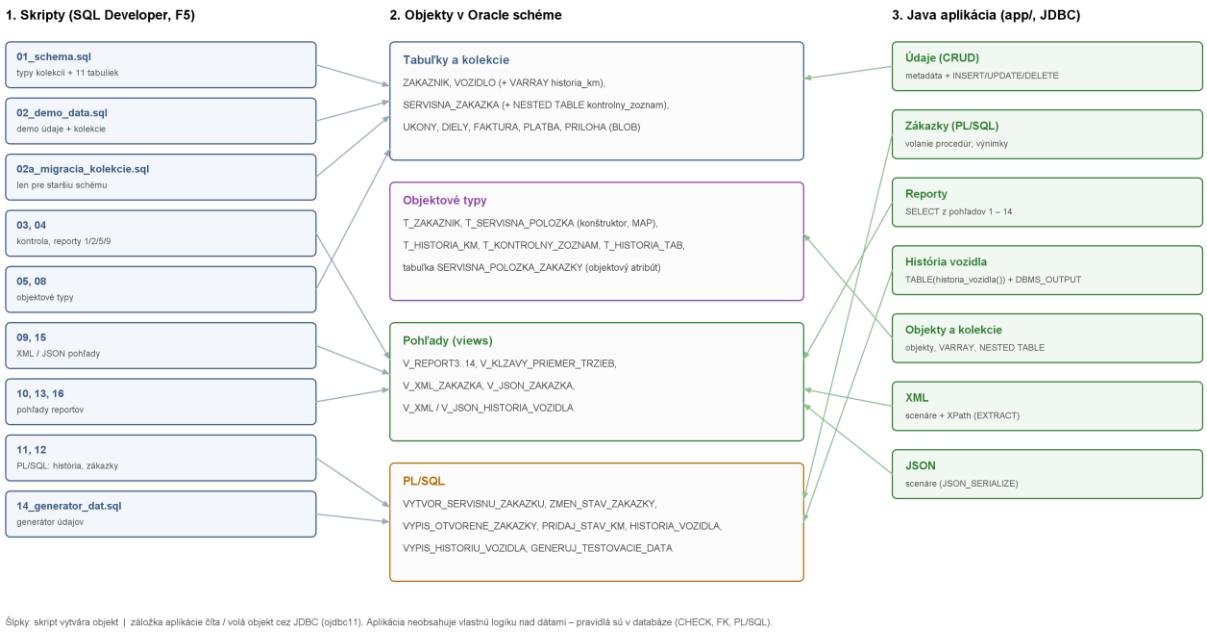
5.3 Spustenie a prihlásenie

1. Windows: dvojklik na `app/spusti.bat`; Linux / macOS: `./app/spusti.sh`; alebo `java -jar app/target/autoservis-pds.jar`.
2. V prihlasovacom okne vyplňte **JDBC URL** `jdbc:oracle:thin:@//oradb.fri.uniza.sk:1521/orclpdb.fri.uniza.sk`, **používateľa** a **heslo** vašej schémy a potvrďte **OK**.
3. URL a meno sa uložia do súboru `db.properties` vedľa aplikácie (heslo nikdy) a pri ďalšom spustení sa predvyplnia. Súbor je v `.gitignore`, do repozitára sa nedostane; vzor je `app/db.properties.example`.

Tip: Ak niektorej záložke chýbajú objekty v databáze, aplikácia nespadne – zobrazí správu, ktorý skript treba spustiť.

6. Ako je všetko prepojené – architektúra

Projekt má tri vrstvy. SQL skripty vytvoria v schéme objekty; aplikácia sa k nim pripája cez JDBC a iba ich používa. Pravidlá (cudzie kľúče, CHECK obmedzenia, prechody stavov zákazky, kontrola tachometra) sú v databáze, preto platia rovnako pre aplikáciu, pre SQL Developer aj pre generátor údajov.



Obrázok 1 – skripty vytvárajú objekty v Oracle, záložky aplikácie ich čítajú a volajú.

6.1 Kto s kým komunikuje

Vrstva	Technológia	Úloha
SQL skripty	SQL, PL/SQL (SQL Developer)	jednorazovo vytvoria tabuľky, typy, pohľady, procedúry a demo dáta
Oracle schéma	tabuľky, CHECK / FK, objektové typy, kolekcie, pohľady, PL/SQL	uchováva dáta a vynucuje pravidlá; pohľady pripravujú reporty a XML / JSON dokumenty
JDBC	ojdbc11, jedno spojenie (`Db.java`), autocommit	prenáša SQL príkazy a výsledky medzi aplikáciou a databázou; všade bind premenné (?)
Aplikácia	Java 17, Swing, 7 záložiek	zobrazuje dáta, volá procedúry, preloží chyby Oracle do slovenčiny (`Chyby.java`)

7. Dátový model

Model má **11 základných tabuliek** a tabuľku `SERVISNA_POLOZKA_ZAKAZKY` s objektovým atribútom – spolu **12 tabuliek a 13 cudzích kľúčov**. Je navrhnutý v 3NF: údaje sa neopakujú, zákazník sa k zákazke dostane cez vozidlo. Pri úkonoch a dieloch sa ukladá cena platná v čase opravy, aby zmena katalógu neovplyvnila históriu.

7.1 Tabuľky a ich vzťahy

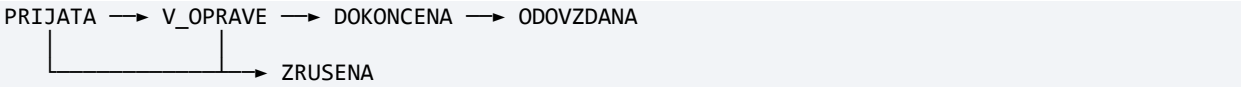
Tabuľka	Význam	Väzby
ZAKAZNIK	zákazník (meno, kontakt, e-mail UNIQUE)	1 : N VOZIDLO
VOZIDLO	vozidlo (VIN, ŠPZ UNIQUE) + **VARRAY historia_km**	N : 1 ZAKAZNIK; 1 : N SERVISNA_ZAKAZKA
ZAMESTNANEC	mechanik / technik (pozícia CHECK, aktívny A/N)	1 : N zákazky a vykonané úkony (voliteľne)
SERVISNA_ZAKAZKA	oprava: dátumy, popis, stav, km + **NESTED TABLE kontrolny_zoznam**	centrum modelu – naň odkazujú úkony, diely, faktúra, prílohy, položky
SERVISNY_UKON	katalóg služieb so štandardnou cenou	1 : N VYKONANY_UKON

Tabuľka	Význam	Väzby
VYKONANY_UKON	úkon v konkrétnej zákazke (množstvo, cena v čase opravy)	N : 1 zákazka, úkon, zamestnanec
NAHRADNY_DIEL	katalóg dielov (katalógové číslo UNIQUE, sklad)	1 : N POUZITY_DIEL
POUZITY_DIEL	diel použitý v zákazke (množstvo, cena v čase opravy)	N : 1 zákazka, diel
FAKTURA	faktúra za zákazku (číslo UNIQUE, splatnosť, suma)	1 : 1 zákazka (UNIQUE FK); 1 : N PLATBA
PLATBA	prijatá platba (spôsob CHECK)	N : 1 FAKTURA – čiastočné úhrady
PRILOHA	súbor k zákazke – **BLOB** (fotka, sken)	N : 1 zákazka
SERVISNA_POLOZKA_ZAKAZKY	rozpis položiek zákazky ako **objekt** T_SERVISNA_POLOZKA	N : 1 zákazka

7.2 Pravidlá, ktoré stráži databáza

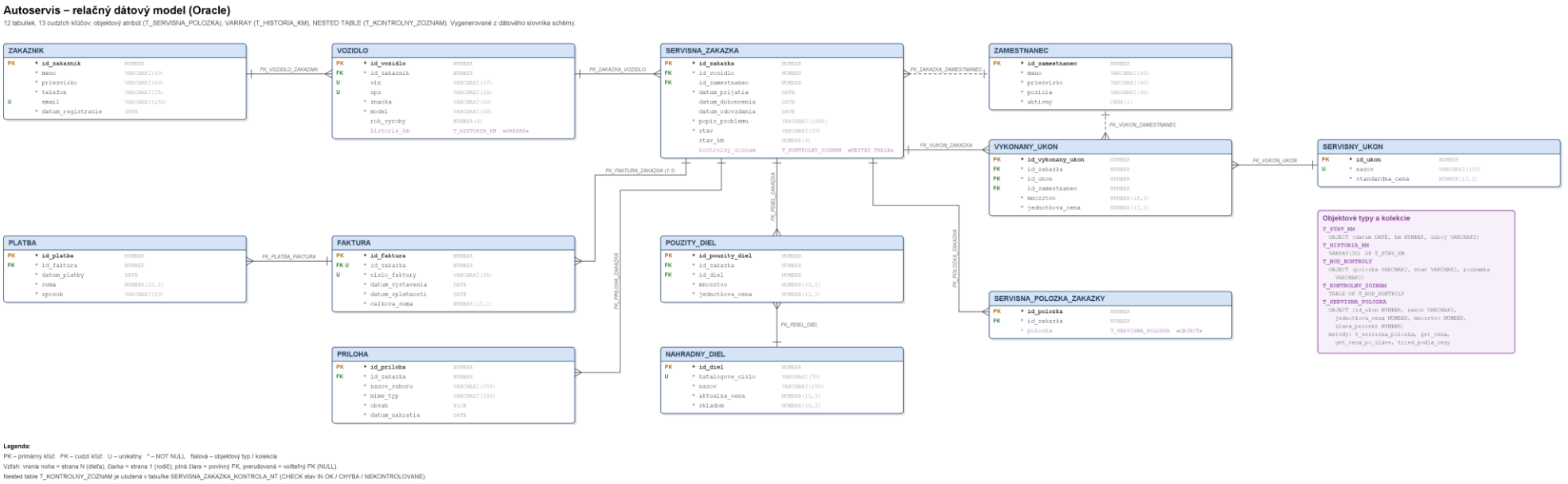
- **Stav zákazky** (`CK_ZAKAZKA_KONZISTENCIA`): PRIJATA / V_OPRAVE / ZRUSENA nemajú dátum dokončenia, DOKONCENA ho má a nemá dátum odovzdania, ODOVZDANA má oba.
- **Dátumy**: dokončenie ≥ prijatie, odovzdanie ≥ dokončenie; splatnosť faktúry ≥ vystavenie.
- **Hodnoty**: ceny a sklad ≥ 0, množstvo > 0, VIN má 17 znakov, rok výroby 1886 – 2100, body kontroly majú stav OK / CHYBA / NEKONTROLOVANE.
- **Jedinečnosť**: e-mail zákazníka, VIN a ŠPZ, katalógové číslo dielu, číslo faktúry, jedna faktúra na zákazku.
- **Tržby vs. faktúry**: fakturovaná suma (FAKTURA) a skutočne prijaté platby (PLATBA) sú odlišné metriky – reporty tržieb počítajú platby.

7.3 Priebeh servisnej zákazky



Pri prijatí sa vytvorí kontrolný zoznam a stav tachometra sa zapíše do histórie vozidla. Počas opravy pribúdajú úkony a diely; dokončiť možno len zákazku s aspoň jedným úkonom. Po dokončení sa vystaví faktúra, po zaplatení sa vozidlo odovzdá. Prechody kontroluje procedúra `ZMEN_STAV_ZAKAZKY` (kapitola 8.2).

7.4 ER diagram



Obrázok 2 – relačný model vygenerovaný z dátového slovníka (`Autoservis_ERD_v2.png`). Vrania noha = strana N, čiarka = strana 1, prerušovaná čiara = voliteľný cudzí kľúč, fialová = objektový typ / kolekcia. Obrázok v plnom rozlíšení je v repozitári.

8. Logika v databáze

8.1 Objektové typy a kolekcie

Typ	Druh	Kde sa používa a načo
T_SERVISNA_POLOZKA	OBJECT – vlastný konštruktor, `get_cena()`, `get_cena_po_zlave()`, MAP `tried_podla_ceny`	stĺpec `polozka` v tabuľke SERVISNA_POLOZKA_ZAKAZKY; konštruktor nastaví množstvo 1 a zľavu 0 %; MAP umožní `ORDER BY p.polozka` a porovnanie `>` podľa ceny po zľave (report 11)
T_ZAKAZNIK	OBJECT s metódami `cele_meno`, `kontakt`, `pocet_vozidiel`	objekty sa vytvárajú priamo v dopyte z tabuľky ZAKAZNIK
T_STAV_KM, T_HISTORIA_KM	OBJECT + **VARRAY(30)**	stĺpec `VOZIDLO.historia_km` – chronologická história tachometra (dátum, km, zdroj STK / SERVIS / PREDAJ / ZAKAZNIK)
T_BOD_KONTROLY, T_KONTROLNY_ZOZNAM	OBJECT + **NESTED TABLE**	stĺpec `SERVISNA_ZAKAZKA.kontrolny_zoznam` – body kontroly pri prijatí; uložené v SERVISNA_ZAKAZKA_KONTROLA_NT s CHECK na stav
T_HISTORIA_ZAZNAM, T_HISTORIA_TAB	OBJECT + **TABLE** (SQL kolekcia)	návratový typ funkcie `HISTORIA_VOZIDLA` – dá sa čítať ako tabuľka `SELECT … FROM TABLE(historia_vozidla(1))`

VARRAY vs. NESTED TABLE: VARRAY má pevné maximum a poradie prvkov a mení sa vždy ako celok (procedúra načíta pole, pridá prvok cez `EXTEND` a zapíše ho späť). Vnorená tabuľka nemá limit a jednotlivé prvky sa dajú meniť priamo SQL príkazmi:

```
INSERT INTO TABLE(SELECT kontrolny_zoznam FROM servisna_zakazka WHERE id_zakazka = 4)
VALUES (t_bod_kontroly('Stierace', 'OK', NULL));

UPDATE TABLE(SELECT kontrolny_zoznam FROM servisna_zakazka WHERE id_zakazka = 4) k
   SET k.stav = 'OK' WHERE k.polozka = 'Karoseria';

SELECT v.spz, h.datum, h.km FROM vozidlo v, TABLE(v.historia_km) h;   -- rozbalenie VARRAY
```

8.2 PL/SQL procedúry a funkcie

Podprogram	Skript	Čo robí
VYTVOR_SERVISNU_ZAKAZKU	12	skontroluje vstupy, vozidlo, zamestnanca a tachometer; vloží zákazku PRIJATA so štandardným kontrolným zoznamom a zapíše km do histórie vozidla; vráti ID (OUT)
ZMEN_STAV_ZAKAZKY	12	zamkne zákazku (`FOR UPDATE`), overí povolený prechod stavu, pri DOKONCENA vyžaduje aspoň jeden úkon, nastaví dátumy
VYPIS_OTVORENE_ZAKAZKY	12	explicitný kurzor nedokončených zákaziek s `%ROWTYPE` a vnorený parametrizovaný kurzor úkonov; výpis cez DBMS_OUTPUT
PRIDAJ_STAV_KM	12	pridá záznam do VARRAY histórie km; kontroluje chronológiu, stočenie tachometra, zdroj; pri plnom poli vyradí najstarší záznam
STANDARDNY_KONTROLNY_ZOZNAM	12	funkcia – vráti 5 štandardných bodov kontroly (NEKONTROLOVANE)
HISTORIA_VOZIDLA	11	funkcia – vráti SQL kolekciu T_HISTORIA_TAB so zákazkami vozidla (BULK COLLECT)
VYPIS_HISTORIU_VOZIDLA	11	textový výpis histórie: PL/SQL RECORD, vnorená tabuľka, asociatívne pole, súhrn (deň / dni / dní)
GENERUJ_TESTOVACIE_DATA	14	generátor konzistentných testovacích údajov vrátane kolekcií

Kódy chýb (RAISE_APPLICATION_ERROR)

Kód	Význam
-20101	nesprávne vstupné údaje (prázdny popis, budúci dátum, nižší tachometer ako pri poslednej zákazke)
-20102 / -20103	vozidlo neexistuje / zamestnanec neexistuje alebo nie je aktívny
-20104 / -20105	servisná zákazka neexistuje / neznámy stav
-20106	nepovolená zmena stavu (napr. ODOVZDANA → V_OPRAVE); v teste odchytená vlastnou výnimkou s `PRAGMA EXCEPTION_INIT`
-20107	zákazku bez vykonaného úkonu nemožno dokončiť
-20111 / -20112 / -20113	história km: starší dátum / nižší stav (stočenie) / neznámy zdroj
-20301	konštruktor T_SERVISNA_POLOZKA – záporná alebo prázdna cena
-20310	vozidlo pre históriu neexistuje (NO_DATA_FOUND preložené na vlastnú chybu)

Aplikácia tieto chyby zachytí a zobrazí len zrozumiteľný text (napr. *Nepovolena zmena stavu zakazky 1: ODOVZDANA → V_OPRAVE*).

8.3 Pohľady a reporty

Väčšina reportov je uložená ako **pohľad** (view). Aplikácia potom posiela jednoduchý `SELECT * FROM pohľad`, a ten istý pohľad sa dá použiť aj v SQL Developeri – logika je na jednom mieste.

№	Report	Kde	Technika
1, 2	celkové a mesačné tržby	04 (priamy SELECT)	SUM, COUNT, GROUP BY, TRUNC
3	TOP 10 zákazníkov	V_REPORT3_TOP_ZAKAZNICI	RANK()
4 / 4b	najčastejšie úkony / úkon mechanika	V_REPORT4_… / dopyt	ROW_NUMBER(), PARTITION BY
5	priemerná dĺžka opravy	04	AVG, rozdiel dátumov
6	porovnanie mesačných tržieb	V_REPORT6_POROVNANIE_MESIACOV	LAG(), kalendár mesiacov
7	kumulatívne tržby	V_REPORT7_KUMULATIVNE_TRZBY	SUM() OVER (… UNBOUNDED PRECEDING)
8	7-dňový kĺzavý priemer	V_KLZAVY_PRIEMER_TRZIEB	AVG() OVER (ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), kalendár dní
9	opravy podľa značky a modelu	04	LEFT JOIN, GROUP BY
10	servisná história vozidla	HISTORIA_VOZIDLA, VYPIS_HISTORIU_VOZIDLA	RECORD, kolekcie, BULK COLLECT
11	detail zákazky (položky)	SERVISNA_POLOZKA_ZAKAZKY	objektový typ, metódy, MAP
12	export zákazky	V_XML_ZAKAZKA, V_JSON_ZAKAZKA	XMLELEMENT…, JSON_OBJECT…
13 / 13b	ročný nájazd / história km	V_REPORT13_…, V_REPORT13B_…	TABLE() nad VARRAY, LAG()
14 / 14b	kontrolný zoznam / zistené chyby	V_REPORT14_…, V_REPORT14B_…	TABLE() nad NESTED TABLE, agregácie

Prečo: Reporty 6 a 8 používajú **kalendár** (generovaný `CONNECT BY LEVEL`), aby mesiace / dni bez platby mali hodnotu 0. Bez neho by okno `ROWS BETWEEN 6 PRECEDING` počítalo 7 riadkov s platbou, nie 7 kalendárnych dní.

8.4 XML a JSON

- **Generovanie z relačných dát** – `XMLELEMENT`, `XMLATTRIBUTES`, `XMLFOREST`, `XMLAGG` (XML) a `JSON_OBJECT`, `JSON_ARRAYAGG`, `JSON_SERIALIZE … PRETTY` (JSON). Zoznamy úkonov, dielov a bodov kontroly sú v samostatných poddotazoch, aby sa riadky navzájom nenásobili.
- **Uloženie a dotazovanie XML** (cvičenie 4) – tabuľka `XML_ZAKAZKY OF XMLTYPE`, dotazy `EXTRACT`, `EXTRACTVALUE`, `EXISTSNODE`, `XMLTABLE` a úprava `UPDATEXML`.
- **Čítanie JSON späť** – `JSON_VALUE`, `JSON_TABLE` (vnorené pole úkonov na riadky) a `JSON_EXISTS` s filtrom.
- **Scenáre**: servisná zákazka (report 12, s kontrolným zoznamom), zákazníci a vozidlá, história vozidla (s históriou km), faktúry a platby so zostatkom.

9. Aplikácia

9.1 Záložky a ich prepojenie na databázu

Záložka	Čo v nej urobíte	Používané objekty
Údaje (CRUD)	vybrať ľubovoľnú z 11 tabuliek, pridať / upraviť (aj dvojklikom) / zmazať záznam, nahrať a uložiť prílohu	všetky tabuľky; metadáta USER_TAB_COLUMNS, USER_CONSTRAINTS
Zákazky (PL/SQL)	založiť zákazku, meniť jej stav, vypísať otvorené zákazky	VYTVOR_SERVISNU_ZAKAZKU, ZMEN_STAV_ZAKAZKY, VYPIS_OTVORENE_ZAKAZKY
Reporty	vybrať report 1 – 9, 13, 14 a zobraziť výsledok aj SQL	pohľady V_REPORT…, V_KLZAVY_PRIEMER_TRZIEB
História vozidla	vybrať vozidlo – tabuľka zákaziek a textový výpis procedúry	HISTORIA_VOZIDLA, VYPIS_HISTORIU_VOZIDLA
Objekty a kolekcie	podzáložky: položky zákazky (objekt), T_ZAKAZNIK, VARRAY história km, NESTED TABLE kontrolný zoznam	SERVISNA_POLOZKA_ZAKAZKY, T_ZAKAZNIK, PRIDAJ_STAV_KM, TABLE()
XML	vybrať scenár a parameter, vygenerovať, vyhodnotiť XPath, uložiť do súboru	V_XML_ZAKAZKA, V_XML_HISTORIA_VOZIDLA, dopyty
JSON	vybrať scenár a parameter, vygenerovať, uložiť	V_JSON_ZAKAZKA, V_JSON_HISTORIA_VOZIDLA, dopyty

9.2 Triedy aplikácie

Trieda	Zodpovednosť
Main	prihlasovacie okno, `db.properties`, hlavné okno so 7 záložkami; obsah záložky sa načíta pri jej otvorení
Db	jediné JDBC spojenie (autocommit); `zachytVystup` – zapne DBMS_OUTPUT, zavolá procedúru a prečíta jej výpis
Chyby	preklad chýb Oracle: ORA-20xxx z procedúr, FK (2291 / 2292), CHECK (2290), UNIQUE (1), NOT NULL (1400)
TabulkyPanel, TabulkaMeta, CrudDialog	všeobecný CRUD: metadáta z dátového slovníka → formulár; FK ako výber, CHECK hodnoty ako výber, BLOB zo súboru, kolekcie len na čítanie
ZakazkyPanel	volanie procedúr zákaziek cez CallableStatement, OUT parameter s ID novej zákazky
ReportyPanel	zoznam reportov (názov, popis, skript, SQL) a zobrazenie výsledku
HistoriaPanel	kolekcia z funkcie + výpis procedúry
ObjektyPanel, PolozkyPanel, ObjectPanel, HistoriaKmPanel, KontrolnyZoznamPanel	objekty a kolekcie – zobrazenie, pridanie konštruktorom, VARRAY a DML nad vnorenou tabuľkou
DokumentyPanel	jedna trieda pre XML aj JSON – scenáre s parametrom (zákazka / vozidlo), XPath cez

Trieda	Zodpovednosť
	`EXTRACT(XMLTYPE(?), ?)`
TypeInstaller	ak chýba typ T_ZAKAZNIK, nainštaluje ho z pribaleného `05_object_type.sql`

9.3 Ako aplikácia pracuje s databázou

- **Bezpečnosť**: hodnoty od používateľa idú vždy cez bind premenné (`?`), nikdy sa nevkladajú do textu SQL. Názvy tabuliek a stĺpcov pochádzajú z pevného zoznamu alebo z dátového slovníka. Heslo sa nikde neukladá.
- **Transakcie**: spojenie má autocommit – každá zmena z aplikácie sa uloží hneď. Procedúry samy COMMIT nerobia, aby ich bolo možné testovať v SQL Developeri s ROLLBACK.
- **Všeobecný CRUD**: `TabulkaMeta` prečíta stĺpce, PK, FK a CHECK `IN (…)`; `CrudDialog` z toho poskladá formulár a príkaz `INSERT` / `UPDATE` iba so stĺpcami, ktoré sa dajú meniť (identity, DEFAULT a kolekcie preskočí).
- **DBMS_OUTPUT**: procedúry s textovým výstupom sa volajú cez `Db.zachytVystup` – po volaní sa riadky vyberú `DBMS_OUTPUT.GET_LINE` v cykle.

10. Priebeh typických scenárov

10.1 Nová zákazka a jej dokončenie

1. Záložka **Zákazky (PL/SQL)**: vyberiete vozidlo, mechanika, dátum, popis a stav km a kliknete **Vytvoriť zákazku**.
2. Aplikácia zavolá `vytvor_servisnu_zakazku(…, :id)`. Procedúra overí vstupy (napr. km nesmie byť nižší ako minule), vloží zákazku v stave PRIJATA so štandardným kontrolným zoznamom a zavolá `PRIDAJ_STAV_KM` – v histórii vozidla pribudne záznam SERVIS.
3. V **Objekty a kolekcie → NESTED TABLE** vyplníte body kontroly (OK / CHYBA + poznámka) – mení sa priamo vnorená tabuľka.
4. V **Údaje (CRUD)** pridáte do VYKONANY_UKON a POUZITY_DIEL úkony a diely (zákazku a úkon vyberiete zo zoznamu).
5. Späť v záložke Zákazky zmeníte stav na **V_OPRAVE → DOKONCENA → ODOVZDANA**. Nepovolený krok (napr. dokončenie bez úkonu) skončí chybou ORA-2010x a zobrazí sa jej text.
6. Výsledok sa hneď prejaví v reportoch, histórii vozidla aj v XML / JSON exporte zákazky.

10.2 Report

Záložka **Reporty** → výber reportu → aplikácia pošle `SELECT … FROM v_report…`, Oracle vyhodnotí pohľad (agregácie, analytické funkcie, okná) a výsledok sa zobrazí v tabuľke spolu s použitým SQL. Ak pohľad chýba, aplikácia oznámi, ktorý skript treba spustiť.

10.3 Export XML s XPath

Záložka **XML** → scenár *Servisná zákazka* → zákazka (alebo všetky) → **Generovať**. Pohľad `V_XML_ZAKAZKA` poskladá dokument; po zadaní XPath, napr. `/zakazky/zakazka/kontrolny_zoznam/bod[@stav="CHYBA"]`, ho databáza vyhodnotí funkciou `EXTRACT` nad práve vygenerovaným dokumentom. **Uložiť do súboru** zapíše dokument ako `.xml`.

11. Riešenie problémov

Problém	Príčina a riešenie
ORA-12541 / ORA-12514	zlý hostiteľ, port alebo názov služby v JDBC URL; skontrolujte URL z kapitoly 5.3
ORA-01017	nesprávne meno alebo heslo
ORA-00942 / záložka hlási chýbajúci skript	v schéme chýbajú objekty – spustite skripty podľa tabuľky v kapitole 4.2
ORA-00904 "HISTORIA_KM" / "KONTROLNY_ZOZNAM"	schéma vznikla starým `01_schema.sql` – spustite `02a_migracia_kolekcie.sql` (kapitola 4.3)
ORA-00955 pri spustení skriptu	objekt už existuje (skript bol spustený) – ignorovať alebo použiť časť UPRATANIE
ORA-02303 pri `11_…sql`	typ histórie už existuje a má závislosti – odkomentovať UPRATANIE na konci skriptu
ORA-01031	chýba právo CREATE TYPE / VIEW / PROCEDURE – požiadať správcu servera
ORA-22908 pri vkladaní bodu kontroly v SQL	kolekcia je NULL – najprv `UPDATE … SET kontrolny_zoznam = t_kontrolny_zoznam()` (aplikácia to robí sama)
JSON scenár zlyhá	Oracle starší ako 19c (chýba JSON_SERIALIZE)
Aplikácia sa nespustí	skontrolujte `java -version` (17+) a či existuje `app/target/autoservis-pds.jar` (`mvn clean package`)

12. Rozdelenie práce v tíme

Člen	Zodpovednosť	Hlavné výstupy
A – člen 1 (AnatoliuLop)	dátový model, databáza, SQL reporty	ER diagram, `01_schema.sql`, `02_demo_data.sql`, `03`, `04` (reporty 1, 2, 5, 9)
M – člen 2 (Matej Paprnák)	PL/SQL a analytické reporty (podľa plánu); odovzdal základ aplikácie	Java aplikácia (prihlásenie, pôvodný CRUD, JSON / XML), `05_object_type.sql`
D – člen 3 (Danyil Berezhnyi)	objekty, XML, integrácia; výstupy 8, 10, 11, 12	`08` – `16`, kolekcie v modeli, `02a`, CRUD všetkých tabuliek, PL/SQL zákaziek, reporty 3 – 8, 13, 14, generátor, reštrukturalizácia aplikácie, ER diagram v2

Podrobný zoznam súborov s autormi a stav plnenia požiadaviek checkpointu a finálneho odovzdania je v `README.md`.